

Katholieke
Universiteit
Leuven

**Faculty of
Engineering Technology
Ghent**

ADVANCED COMPUTER ARCHITECTURE

Lab 3: Warps, Thread divergences and striding

Tom Thys

Contents

1 Introduction **2**

2 Image Inversion **2**
2.1 Results 2

3 Thread Divergence **3**
3.1 Results 3

4 Thread Divergence with planar input **3**
4.1 Planar Results 4

5 Comparison **4**

A Description: reverse.cu **5**

1 Introduction

In this lab, we investigated how the number of threads per block influences the overall performance of a GPU kernel. Since each warp consists of 32 threads, the number of warps per block directly depends on the chosen block size. This relationship plays an important role in how efficiently GPU resources are utilized.

The goal of the lab was to analyze how varying the number of threads (and thus warps) per block affects the computation time of an image-processing kernel. Specifically, we explored how different configurations can lead to overprovisioning of GPU cores or, conversely, to underutilization of the available resources. We also examined cases where thread divergence occurs (situations where threads within the same warp follow different execution paths) which can lead to idle cores and reduced parallel efficiency.

By timing the kernel execution across multiple configurations, we were able to observe the trade-off between warp-level parallelism and execution efficiency. The results demonstrate how proper grid and block partitioning, together with minimizing divergence, are essential to achieving optimal GPU performance.

2 Image Inversion

In the first part of the lab, the goal was to implement a CUDA kernel that performs image inversion. For each pixel, the RGB color components are inverted according to the formula:

$$x' = 255 - x, \quad \text{where } x \in [0, 255].$$

The kernel was designed so that each thread processes one pixel. To analyze GPU performance, the number of threads per block was varied while measuring the execution time. Since each block is divided into warps of 32 threads, changing the block size directly influences how many warps are launched and how efficiently the GPU resources are utilized.

By running the kernel with different configurations, we were able to observe how the number of threads per block—and therefore the number of warps—affects computation time. In general, performance improves with increased parallelism up to the point where GPU resources become saturated.

2.1 Results

As shown in Table 1, the execution time decreases significantly as the number of threads increases. This trend continues up to around 128 threads per block, after which the performance gain becomes negligible. The reason for this behavior is that increasing the number of threads initially improves parallelism, allowing more pixels to be processed concurrently. However, beyond a certain point, the GPU resources (such as the number of available cores and warp schedulers) become fully utilized, and additional threads no longer contribute to higher throughput.

This saturation effect demonstrates that optimal performance is achieved when the GPU's computational capacity is fully occupied but not oversubscribed. In this case, configurations of 128 or 256 threads per block yield the shortest execution times, indicating a balance between occupancy and scheduling overhead.

Threads	**Time (ms)**
16	1.79084
32	0.897111
64	0.370791
128	0.340083
256	0.343687
512	0.355998
1024	0.381934

Table 1: Execution time for varying thread to reverse image colors over 1000 runs

3 Thread Divergence

In the second part of the lab, the image inversion kernel was extended to introduce *thread divergence*. Specifically, instead of simply inverting all color channels, the red component (R) was processed using a one-dimensional Gaussian blur, while the green (G) and blue (B) components were still inverted as before. The Gaussian blur was defined as:

$$r_i = 0.1r_{i-2} + 0.25r_{i-1} + 0.5r_i + 0.25r_{i+1} + 0.1r_{i+2},$$

where r_i represents the intensity of the red pixel at position i. This operation introduces conditional branching in the kernel since only one color channel follows a different computation path, leading to divergent execution within warps.

Thread divergence occurs because threads in the same warp no longer execute the same instructions simultaneously. When part of a warp follows the Gaussian computation and the rest performs simple inversion, the GPU must serialize the different execution paths. As a result, some threads become idle while others complete their tasks, reducing the effective parallel efficiency.

By comparing the execution times between the divergent and non-divergent kernels, we observed a clear increase in computation time when divergence was introduced. This confirms that minimizing divergence is crucial to maintaining high throughput. Reorganizing data so that threads within a warp process similar operations (e.g., separating the color channels into distinct memory regions) can mitigate this performance loss.

3.1 Results

As shown in Table 2, the Gaussian blur kernel exhibits a similar trend to the inversion kernel: increasing the number of threads initially reduces the execution time due to higher parallelism. However, the overall execution times are slightly higher compared to the non-divergent case. This increase in computation time results from thread divergence, as only the red component undergoes the Gaussian blur while the other components are inverted.

Once again, performance improvements plateau around 128 threads per block, indicating that the GPU reaches optimal occupancy at this configuration. Beyond this point, adding more threads no longer reduces the execution time and may even introduce minor overhead due to scheduling and memory access patterns. These results confirm that minimizing thread divergence and carefully selecting block sizes are key to achieving efficient GPU utilization.

Threads	**Time (ms)**
16	2.04047
32	1.07531
64	0.636736
128	0.634645
256	0.658149
512	0.738761
1024	0.865300

Table 2: Execution time for varying thread counts to gaussian blur image colors over 1000 runs

4 Thread Divergence with planar input

As an additional, non-obligatory experiment, we modified the kernel to process the image in a *planar* format instead of the standard interleaved RGB layout. In the interleaved layout, pixel values are stored sequentially as `RGBRGBRGB...`, meaning consecutive threads within a warp operate on different color channels. This often causes thread divergence, since the red component undergoes the Gaussian blur while the green and blue components are simply inverted.

In the planar layout, the image data is reorganized into three separate memory regions: `RR...R`, `GG...G`, and `BB...B`. This ensures that threads within a warp process the same color component, eliminating divergence caused by branching between different color computations.

The modified kernel showed a small but measurable performance improvement compared to the interleaved version. Although the total amount of computation remains the same, the absence of divergence allows warps

to execute more efficiently, with fewer idle threads. This experiment highlights how proper data organization can have a significant impact on GPU performance, particularly when different processing steps are applied to different data subsets.

4.1 Planar Results

As shown in Table 3, the planar image layout achieves slightly better performance than the interleaved RGB format. The execution times are consistently lower, especially for higher thread counts. This improvement can be attributed to the elimination of thread divergence: all threads within a warp now process the same color channel, allowing them to execute the same instructions in parallel without branching.

The best performance is again observed around 64 to 128 threads per block, after which the execution time stabilizes. This confirms that while computational workload remains identical, the planar data organization leads to more coherent memory access patterns and fully synchronized warp execution, resulting in improved overall efficiency.

Threads	Time (ms)
16	1.9287
32	0.995494
64	0.472641
128	0.479336
256	0.480222
512	0.534172
1024	0.622259

Table 3: Execution time for varying thread counts to Gaussian blur with planar input over 1000 runs

5 Comparison

To clearly visualize the performance differences between the three CUDA kernels, Figure 1 presents a comparison of their execution times for varying thread counts. The plot highlights how each kernel scales with increased parallelism and how thread divergence and data organization influence performance. While all kernels benefit from higher thread counts up to a certain point, the non-divergent inversion kernel consistently achieves the lowest execution time. The Gaussian blur kernel with a planar image layout performs slightly better than its interleaved counterpart, confirming that minimizing divergence and improving memory access patterns enhance GPU efficiency.

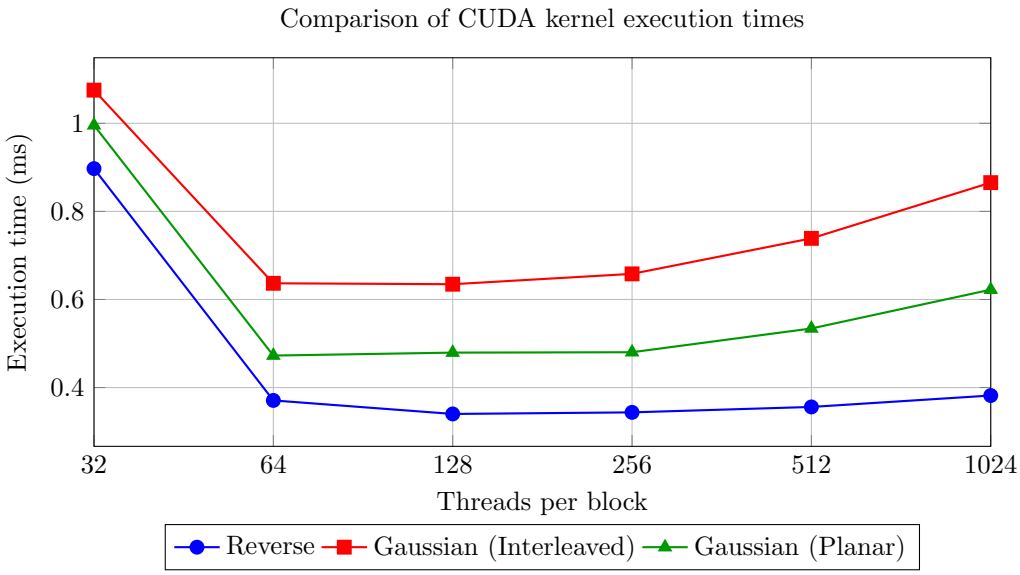


Figure 1: Execution time comparison for different CUDA kernels using varying thread counts over 1000 runs

A Description: reverse.cu

Listing 1: reverse.cu

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations

// #define M 512       // Lenna width
// #define N 512       // Lenna height
#define M 960       // VR width
#define N 1280       // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length


__global__ void reverse_with_stride(uint8_t* input_array, uint8_t* output_array, int m, int n, int c){
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = idx; i < m*n*c; i += stride) {
        output_array[i] = 255 - input_array[i];
    }
}

__global__ void reverse(uint8_t* input_array, uint8_t* output_array, int m, int n, int c){
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    if (idx < m*n*c){
        output_array[idx] = 255 - input_array[idx];
    }
}




__global__ void gaussian_blur(uint8_t* input_array, uint8_t* output_array, int m, int n, int c) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int total = m * n * c;

    if (idx < total) {
        int pixel_idx = idx / c;   // pixel number
        int channel   = idx % c;   // 0 = R, 1 = G, 2 = B

        if (channel == 0) { // Red channel: apply 1D Gaussian blur
            // We assume each row is contiguous (row-major order)
            int row = pixel_idx / n;
            int col = pixel_idx % n;

            float r_sum = 0.0f;

            // Compute offsets only along the row
            for (int k = -2; k <= 2; k++) {
                int neighbor_col = col + k;
                if (neighbor_col >= 0 && neighbor_col < n) {
                    int neighbor_idx = (row * n + neighbor_col) * c + 0; // red channel
                    float weight;
                    if (abs(k) == 2) weight = 0.1f;
                    else if (abs(k) == 1) weight = 0.25f;
                    else weight = 0.5f;
                    r_sum += weight * input_array[neighbor_idx];
                }
            }

            output_array[idx] = static_cast<uint8_t>(r_sum);
```

```
67
68          } else {
69              // Green and Blue channels: inversion
70              output_array[idx] = 255 - input_array[idx];
71          }
72      }
73  }
74
75
76  __global__ void gaussian_blur_planar(uint8_t* input_array, uint8_t* output_array, int m, int
        n, int c) {
77      int idx = threadIdx.x + blockIdx.x * blockDim.x;
78      int num_pixels = m * n;
79      int total = num_pixels * c;
80
81      if (idx < total) {
82          int channel = idx / num_pixels;    // 0 = R, 1 = G, 2 = B
83          int pixel_idx = idx % num_pixels;  // pixel index within channel
84
85          int row = pixel_idx / n;
86          int col = pixel_idx % n;
87
88          // Base pointer for this channel
89          uint8_t* in_plane  = input_array  + channel * num_pixels;
90          uint8_t* out_plane = output_array + channel * num_pixels;
91
92          if (channel == 0) {
93              // --- Gaussian blur on RED channel ---
94              float r_sum = 0.0f;
95
96              for (int k = -2; k <= 2; k++) {
97                  int neighbor_col = col + k;
98                  if (neighbor_col >= 0 && neighbor_col < n) {
99                      int neighbor_idx = row * n + neighbor_col;
100                     float weight;
101                     if (abs(k) == 2) weight = 0.1f;
102                     else if (abs(k) == 1) weight = 0.25f;
103                     else weight = 0.5f;
104                     r_sum += weight * in_plane[neighbor_idx];
105                 }
106             }
107
108             out_plane[pixel_idx] = static_cast<uint8_t>(r_sum);
109         }
110         else {
111             // --- Green & Blue: Inversion ---
112             out_plane[pixel_idx] = 255 - in_plane[pixel_idx];
113         }
114     }
115 }
116
117
118
119
120 uint8_t* cuda_stuff(uint8_t* input_array){
121
122     std::cout << N << " " << M;
123
124     uint8_t *in_a, *out_a;
125     int total = M * N * C;
126
127     cudaMalloc((void**)&in_a, N*M*C * sizeof(uint8_t));
128     cudaMalloc((void**)&out_a, N*M*C * sizeof(uint8_t));
129
130
131     // Copy data from host to device
132     cudaMemcpy(in_a, input_array, N * M * C * sizeof(uint8_t), cudaMemcpyHostToDevice);
133
134     reverse<<<(total + 1024 - 1) / 1024, 1024>>>(in_a, out_a, M, N, C);
135
136     int iterations = 1000;
137     std::cout << "Threads\tBlocks\tTime (ms)\n";
138     for (int n_threads = 1; n_threads <= 1024; n_threads *= 2) {
```

```
        for (int n_blocks = 2; n_blocks <= 256 && n_blocks <= (total + n_threads - 1) / n_threads; n_blocks *= 2){
            // int n_blocks = (total + n_threads - 1) / n_threads; // No-striding
            // Create CUDA events for timing
            cudaEvent_t start, stop;
            cudaEventCreate(&start);
            cudaEventCreate(&stop);

            cudaDeviceSynchronize();

            // Record start event
            cudaEventRecord(start);

            for (int i = 0; i < iterations; ++i) {
                // gaussian_blur<<<n_blocks, n_threads>>>(in_a, out_a, M, N, C);
                // gaussian_blur<<<n_blocks, n_threads>>>(in_a, out_a, M, N, C); //for part 2
                //gaussian_blur_planar<<<n_blocks, n_threads>>>(in_a, out_a, M, N, C);
                reverse_with_stride<<<n_blocks, n_threads>>>(in_a, out_a, M, N, C);

            }

            // Record stop event
            cudaEventRecord(stop);
            cudaEventSynchronize(stop);

            float elapsed_ms = 0.0f;
            cudaEventElapsedTime(&elapsed_ms, start, stop);

            // Average time per iteration
            float avg_time = elapsed_ms / iterations;

            std::cout << n_threads << "\t" << n_blocks << "\t" << avg_time << "\n";

            // Cleanup
            cudaEventDestroy(start);
            cudaEventDestroy(stop);
        }
    }

    // Copy result back to host

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C * sizeof(uint8_t));

    cudaMemcpy((void*)new_image_array, (const void*)out_a, N*M*C * sizeof(uint8_t), cudaMemcpyDeviceToHost);

    cudaFree(out_a);
    cudaFree(in_a);

    return new_image_array;
}

uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./lab3/anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
```

```
209     }
210 
211     // Initialize empty image array
212     uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);
213 
214     // Read the image
215     fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);
216 
217     // Close the file
218     fclose(imageFile);
219 
220     // Move the starting pointer and return the flattened image array
221     return image_array + OFFSET;
222 }
223 
224 uint8_t* rearrange_to_RRGGBB(uint8_t* rgb_array) {
225     size_t num_pixels = M * N;
226     uint8_t* planar = (uint8_t*)malloc(num_pixels * C * sizeof(uint8_t));
227     if (!planar) {
228         perror("ERROR: Memory allocation failed");
229         exit(EXIT_FAILURE);
230     }
231 
232     uint8_t* R = planar;
233     uint8_t* G = planar + num_pixels;
234     uint8_t* B = planar + 2 * num_pixels;
235 
236     for (size_t i = 0; i < num_pixels; i++) {
237         R[i] = rgb_array[3 * i + 0];
238         G[i] = rgb_array[3 * i + 1];
239         B[i] = rgb_array[3 * i + 2];
240     }
241 
242     return planar;  // now planar = [R...R G...G B...B]
243 }
244 
245 uint8_t* planar_to_interleaved(uint8_t* planar) {
246     size_t num_pixels = M * N;
247     uint8_t* interleaved = (uint8_t*)malloc(num_pixels * C * sizeof(uint8_t));
248     uint8_t* R = planar;
249     uint8_t* G = planar + num_pixels;
250     uint8_t* B = planar + 2 * num_pixels;
251 
252     for (size_t i = 0; i < num_pixels; i++) {
253         interleaved[3*i + 0] = R[i];
254         interleaved[3*i + 1] = G[i];
255         interleaved[3*i + 2] = B[i];
256     }
257     return interleaved;
258 }
259 
260 
261 
262 void save_image_array(uint8_t* image_array){
263     /*
264      * Save the data of an (RGB) image as a pixel map.
265      *
266      * Parameters:
267      *  - param1: The data of an (RGB) image as a 1D array
268      *
269      */
270     // Try opening the file
271     FILE *imageFile;
272     imageFile=fopen("./lab3/reverse_output_image.ppm","wb");
273     if(imageFile==NULL){
274         perror("ERROR: Cannot open output file");
275         exit(EXIT_FAILURE);
276     }
277 
278     // Configure the file
279     fprintf(imageFile,"P6\n");               // P6 filetype
280     fprintf(imageFile,"%d %d\n", M, N);      // dimensions
281     fprintf(imageFile,"255\n");              // Max pixel
```

```
282
283     // Write the image
284     fwrite(image_array, 1, M*N*C, imageFile);
285
286     // Close the file
287     fclose(imageFile);
288 }
289
290
291 int main (void) {
292
293     // Read the image
294     uint8_t* image_array = get_image_array();
295
296
297     uint8_t* new_image_array = cuda_stuff(rearrange_to_RRGGBB(image_array));
298
299
300     // Save the image
301     save_image_array(planar_to_interleaved(new_image_array));
302
303     return 0;
304 }
```